Exercise 10 - 19.01./20.01.2026

Learning goals

We want to practice E2E testing and performance measurements in Angular.

Playwright (by Microsoft) is an established E2E testing framework that is capable to be used across any frontend technologies. Compared to Cypress it is more stable and has more capabilities (I recently supervised a Master thesis that analysed that in depth for the use-case of microservices).

Practical Exercises

1) Install Playwright in your angular project: ***npm install --save-dev @playwright/test***

and run ***npx playwright install*** to install additional browser drivers.

Create a folder named /tests/playwright at root (next to /src) and add a config „***playwright.config.ts***“ in your project root (next to package.json) to mark tests only in one special folder as playwright tests:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: 'tests/playwright'
});
```

Create a package.json script to run tests in UI mode (skip --ui for headless):

```
"test:e2e": "playwright test --config=playwright.config.ts --ui"
```

Assuming your Angular app is running, you could run tests now if they are located in the tests/playwright folder.

Now add a playwright E2E test file ***login.spec.ts*** that tests navigating to your login page, locates the input, button and checkbox elements, ensures those elements are visible, fills out the form, submits the enabled submit button and ensures we get redirected to /dashboard after form submission.

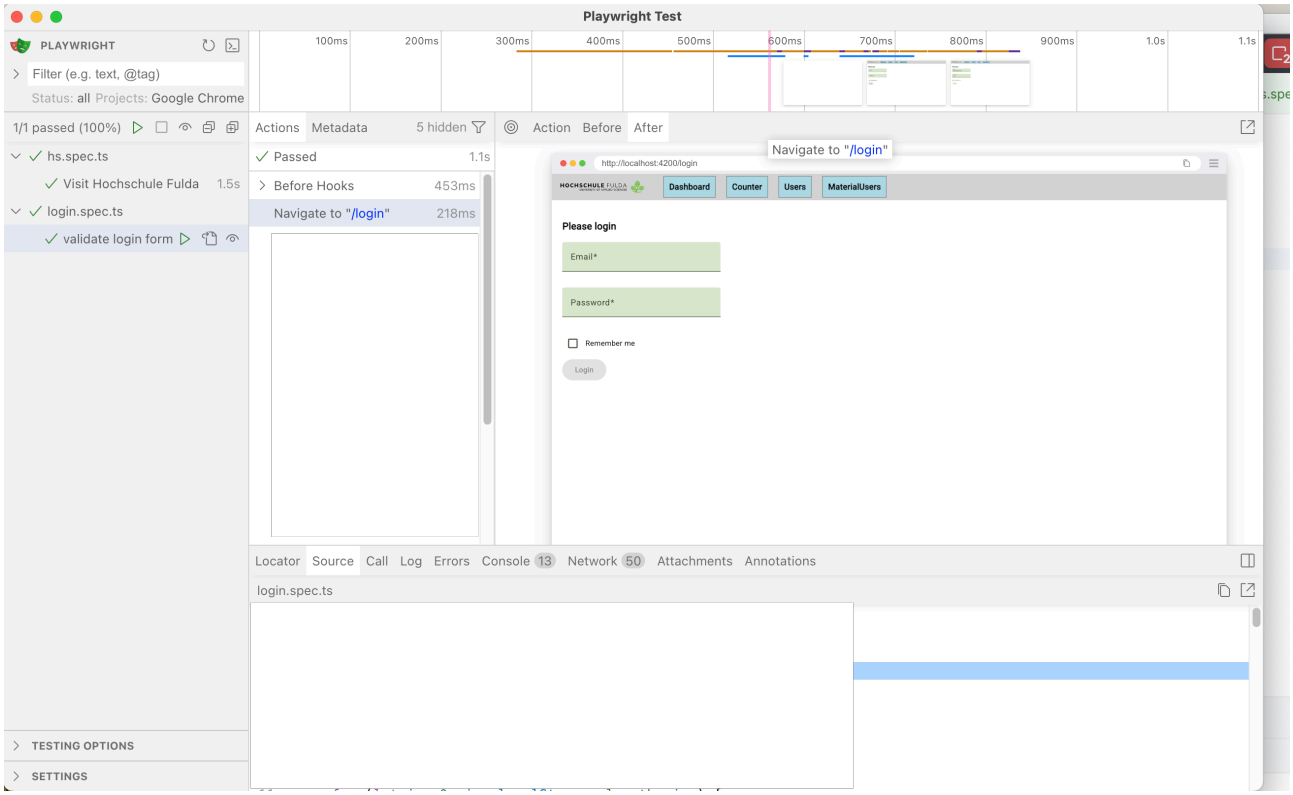
You will need:

- `import { test, expect } from '@playwright/test';`
- ***page.goto()***
- ***page.locator()***
- ***click()***
- ***fill()***
- ***page.url()***

Check the Playwright docu how to write tests if you get stuck:

https://playwright.dev/docs/writing-tests

The playwright UI will help you debugging:



2) Write an E2E Test for https://www.hs-fulda.de/ by creating another hs.spec.ts file (can be existing in same folder).

- Navigate to the HS website using page.goto
- Ensure the cookie banner is clicked away using page.click

- Navigate between screens using a simulated header link click. E.g.: click on „STUDIENANGEBOT“.
- Check that the resulting URL is the one expected (using page.url) and test visibility / existence of the form element that allows searching for a study subject (using page.$$ e.g.).

3) Use a Chrome Browser and visit https://www.hs-fulda.de/

- Analyse CWV using the „**Performance**“ Tab in the Browser Dev Tools
  - What properties are displayed? Is there a „poor“ property?
  - Repeat the measurement simulating 3G network. Is there a „poor“ property?
  - Repeat the analysis for https://www.hs-fulda.de/studium/ihre-bewerbung with no network throttling and throttling to 3G. Are there any „poor“ properties?
- What could be done to improve the poor score?
- Analyse both URLs using the „**Lighthouse**“ Tab in the Chrome Browser Dev Tools. Perform a scan both for mobile and desktop and check recommendations.
  - Is there a difference between mobile and desktop?
  - What metrics feed into the performance score?
  - What are the two top recommendations to improve mobile performance?
  - Find out a potential SEO improvement by checking the recommendations